OS Banana Problem U3

Yash Tadi

PES2UG24CS608

Section - J

Get Your Bearings

```
608$ free -m
              total        used        free      shared  buff/cache   available
Mem:           5785        1131        3934          11         955        4654
Swap:          2134           0        2134

608$ getconf PAGESIZE
4096

608$ awk '/MemTotal/ {print $2 / 4}' /proc/meminfo
1481173

608$
```

5785 MB = 5785*1024 KB = 5923840 KB

4096 bytes = 4 KB

Frames = 5923840/4 = **1480960**

The Virtual Address Space

```
608$ gcc -o layout layout.c -no-pie

608$ ./layout
Text  (main):        0x401186
Rodata:              0x402008
Data  (init):        0x404048
BSS   (uninit):      0x40405c
Heap  (malloc):      0x330922a0
Stack (local):       0x7ffedc38e1d4

Heap-to-Stack gap: 134212242 MB

--- /proc/self/maps (filtered) ---
00400000-00401000 r--p 00000000 08:01 958562                     /home/yashtadi/layout
00401000-00402000 r-xp 00001000 08:01 958562                     /home/yashtadi/layout
00402000-00403000 r--p 00002000 08:01 958562                     /home/yashtadi/layout
00403000-00404000 r--p 00002000 08:01 958562                     /home/yashtadi/layout
00404000-00405000 rw-p 00003000 08:01 958562                     /home/yashtadi/layout
33092000-330b3000 rw-p 00000000 00:00 0                          [heap]
7ffedc370000-7ffedc391000 rw-p 00000000 00:00 0                  [stack]

608$
```

Q1.1:

401186 < 402008 < 404048 < 40405C < Heap << stack

Hence verified

Q1.2:

The heap has permissions rw-p, meaning readable and writable but not executable.

It is not executable to prevent security issues like executing malicious code (buffer overflow attacks).

Q1.3

```
608$ gcc -o layout_pie layout.c

608$ ./layout_pie
Text  (main):        0x56393b8d7199
Rodata:              0x56393b8d8008
Data  (init):        0x56393b8da048
BSS   (uninit):      0x56393b8da05c
Heap  (malloc):      0x5639573c32a0
Stack (local):       0x7ffd0086c764

Heap-to-Stack gap: 43793044 MB

--- /proc/self/maps (filtered) ---
56393b8d6000-56393b8d7000 r--p 00000000 08:01 958583                     /home/yashtadi/layout_pie
56393b8d7000-56393b8d8000 r-xp 00001000 08:01 958583                     /home/yashtadi/layout_pie
56393b8d8000-56393b8d9000 r--p 00002000 08:01 958583                     /home/yashtadi/layout_pie
56393b8d9000-56393b8da000 r--p 00002000 08:01 958583                     /home/yashtadi/layout_pie
56393b8da000-56393b8db000 rw-p 00003000 08:01 958583                     /home/yashtadi/layout_pie
5639573c3000-5639573e4000 rw-p 00000000 00:00 0                          [heap]
7ffd0084d000-7ffd0086e000 rw-p 00000000 00:00 0                          [stack]

608$ ./layout_pie
Text  (main):        0x55b84a673199
Rodata:              0x55b84a674008
Data  (init):        0x55b84a676048
BSS   (uninit):      0x55b84a67605c
Heap  (malloc):      0x55b88357e2a0
Stack (local):       0x7ffff3830644

Heap-to-Stack gap: 44332802 MB

--- /proc/self/maps (filtered) ---
55b84a672000-55b84a673000 r--p 00000000 08:01 958583                     /home/yashtadi/layout_pie
55b84a673000-55b84a674000 r-xp 00001000 08:01 958583                     /home/yashtadi/layout_pie
55b84a674000-55b84a675000 r--p 00002000 08:01 958583                     /home/yashtadi/layout_pie
55b84a675000-55b84a676000 r--p 00002000 08:01 958583                     /home/yashtadi/layout_pie
55b84a676000-55b84a677000 rw-p 00003000 08:01 958583                     /home/yashtadi/layout_pie
55b88357e000-55b88359f000 rw-p 00000000 00:00 0                          [heap]
7ffff3811000-7ffff3832000 rw-p 00000000 00:00 0                          [stack]

608$
```

With PIE enabled the addresses change between runs due to ASLR but without PIE, addresses remain fixed. ASLR improves security by making memory locations unpredictable.

Demand Paging

```
608$ ./demand

[1. before malloc]
VmSize:     2568 kB
VmRSS:      1648 kB
VmSwap:        0 kB

[2. after malloc, before touch - virtual only!]
VmSize:   264716 kB
VmRSS:      1652 kB
VmSwap:        0 kB

[3. after touching 64 MB]
VmSize:   264716 kB
VmRSS:     68544 kB
VmSwap:        0 kB

[4. after touching all 256 MB]
VmSize:   264716 kB
VmRSS:    263796 kB
VmSwap:        0 kB

608$
```

Q2.1:

After malloc, VmSize jumps up from 2568 Kb to 264716 Kb , which is significant compared to negligible change in VmRSS. This happens because no physical memory is allocated until the pages are accessed.

Q2.2:

The expected number of page faults is approximately 65536, because each 4 KB page causes one page fault on first access. The actual count can be slightly higher due to additional faults from code pages, libc libraries, stack pages, and process metadata.

Q2.3:

Right after malloc(256 MB), the process VmSize increases by 256 MB, but the RSS does not, because physical memory is allocated lazily using demand paging only when pages are accessed.

Page Tables

```
608$ sudo ./pagemap
[sudo] password for yashtadi:
=== Before touching any page ===
  page[0]:       VA=0x55e1586592a0  present=1  swapped=0  PFN=0x15eb74  →  PA=0x15eb74000
  page[1]:       VA=0x55e15865a2a0  present=0  swapped=0  (no physical frame assigned)
  page[2]:       VA=0x55e15865b2a0  present=0  swapped=0  (no physical frame assigned)

=== After touching pages 0 and 1 ===
  page[0]:       VA=0x55e1586592a0  present=1  swapped=0  PFN=0x15eb74  →  PA=0x15eb74000
  page[1]:       VA=0x55e15865a2a0  present=1  swapped=0  PFN=0x16a91b  →  PA=0x16a91b000
  page[2]:       VA=0x55e15865b2a0  present=0  swapped=0  (no physical frame assigned)

608$
```

Q3.1:

Before accessing memory, page[1] and page[2] have present = 0, meaning they are not in physical memory. However, page[0] already has present = 1. After touching pages 0 and 1, both pages have present = 1, indicating they are mapped to physical frames. Page[2] remains unmapped since it was never accessed.

Q3.2:

Page 2 is not present (present = 0) and has no PFN. This shows that malloc() only reserves virtual memory , physical memory is allocated only when the page is accessed.

Q3.3:

PFNs differ between runs because the operating system dynamically assigns available physical frames. The allocation is non-deterministic and depends on the current state of memory usage, so different runs may map pages to different physical frames.

Copy on Write

```
608$ ./cow
Parent: allocated and touched 128 MB (32768 pages)

Child READ-ONLY scan:
  Page faults: 5  (should be ~0, pages are shared)

Child WRITE pass:
  Page faults: 32769
  Expected:    32768  (one COW fault per page)

608$
```

Q4.1:
The read-only scan causes ~0 page faults because both parent and child share the same physical pages after fork(). Since no writes occur, no COW is triggered, so the existing mappings are reused without faults.

Q4.2:
The write pass causes 32679 faults (one per page), since each write triggers a COW fault. The extra faults may have come from page table setup, stack/heap accesses, or other minor kernel activity.

Q4.3:
Without COW, a full deep copy would require an extra 128 MB of physical memory. COW is crucial for fork() + exec() because the child usually replaces its memory, so copying everything upfront would be wasteful.

Shared Libraries

```
608$ LD_DEBUG=libs ./hello_dynamic 2>&1 | head -30

      5431:     find library=libc.so.6 [0]; searching
      5431:      search cache=/etc/ld.so.cache
      5431:       trying file=/lib/x86_64-linux-gnu/libc.so.6
      5431:
      5431:
      5431:     calling init: /lib64/ld-linux-x86-64.so.2
      5431:
      5431:
      5431:     calling init: /lib/x86_64-linux-gnu/libc.so.6
      5431:
      5431:
      5431:     initialize program: ./hello_dynamic
      5431:
      5431:
      5431:     transferring control: ./hello_dynamic
      5431:
      5431:
      5431:     calling fini:  [0]
      5431:
      5431:
      5431:     calling fini: /lib/x86_64-linux-gnu/libc.so.6 [0]
      5431:
      5431:
      5431:     calling fini: /lib64/ld-linux-x86-64.so.2 [0]
      5431:
Hello, shared world!

608$
```

Q5.1:

hello_dynamic is much smaller(16K), while hello_static is significantly larger(741K) because it embeds the entire libc inside the binary. Dynamic binaries only store references, whereas static binaries include full library code, increasing size drastically.

```
608$ ls -lh hello_dynamic hello_static

-rwxrwxr-x 1 yashtadi yashtadi  16K Apr  3 19:49 hello_dynamic
-rwxrwxr-x 1 yashtadi yashtadi 741K Apr  3 19:49 hello_static
```

Q5.2:

```
608$ cat /proc/self/maps | grep libc
7fb7c7180000-7fb7c71a8000 r--p 00000000 08:01 1965188                    /usr/lib/x86_64-linux-gnu/libc.so.6
7fb7c71a8000-7fb7c730d000 r-xp 00028000 08:01 1965188                    /usr/lib/x86_64-linux-gnu/libc.so.6
7fb7c730d000-7fb7c7363000 r--p 0018d000 08:01 1965188                    /usr/lib/x86_64-linux-gnu/libc.so.6
7fb7c7363000-7fb7c7367000 r--p 001e2000 08:01 1965188                    /usr/lib/x86_64-linux-gnu/libc.so.6
7fb7c7367000-7fb7c7369000 rw-p 001e6000 08:01 1965188                    /usr/lib/x86_64-linux-gnu/libc.so.6
```

- r-xp → executable code (shared across processes)
- r--p → read-only data (constants, strings)
- rw-p → writable data (process-private globals)

These permissions reflect how different parts of libc are mapped in memory.

Q5.3:

The OS keeps one shared copy of libc's code segment in RAM for all processes. Writable data is not shared—each process gets its own private copy to maintain isolation.

Under The Hood

```
608$ ./alloc_trace

Initial break:       0x55ffaabe0000

After malloc(1KB):   break=0x55ffaac01000  ptr=0x55ffaabe06b0  (break moved up)
After malloc(256KB): break=0x55ffaac01000  ptr=0x7f02f1351010  (break unchanged!)
After malloc(512B):  break=0x55ffaac01000  ptr=0x55ffaabe0ac0

small  at 0x55ffaabe06b0
small2 at 0x55ffaabe0ac0  (close to small - both on heap)
large  at 0x7f02f1351010  (far away - mmap'd separately)
```

```
608$ strace -e brk,mmap ./alloc_trace 2>&1 | tail -20
mmap(NULL, 123371, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fbad1351000
mmap(NULL, 2055800, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fbad115b000
mmap(0x7fbad1183000, 1462272, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7fbad1183000
mmap(0x7fbad12e8000, 352256, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x18d000) = 0x7fbad12e8000
mmap(0x7fbad133e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e2000) = 0x7fbad133e000
mmap(0x7fbad1344000, 52856, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fbad1344000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fbad1158000
brk(NULL)                               = 0x55c12d9d6000
brk(0x55c12d9f7000)                     = 0x55c12d9f7000
mmap(NULL, 266240, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fbad1117000
Initial break:       0x55c12d9d6000

After malloc(1KB):   break=0x55c12d9f7000  ptr=0x55c12d9d72b0  (break moved up)
After malloc(256KB): break=0x55c12d9f7000  ptr=0x7fbad1117010  (break unchanged!)
After malloc(512B):  break=0x55c12d9f7000  ptr=0x55c12d9d76c0

small  at 0x55c12d9d72b0
small2 at 0x55c12d9d76c0  (close to small - both on heap)
large  at 0x7fbad1117010  (far away - mmap'd separately)
+++ exited with 0 +++

608$
```

Q6.1

No, the program break does not move after malloc(256 KB) because large allocations use mmap(). mmap() creates a separate memory region instead of extending the heap via brk().

Q6.2

Small allocations (like 1 KB, 512 B) trigger brk(), while large allocations (256 KB) trigger mmap(). glibc typically switches from brk to mmap at around 128 KB (default threshold).

Q6.3

With only brk(), freeing middle blocks causes fragmentation since the heap can't shrink backward. mmap() avoids this by allocating large blocks separately, allowing them to be individually released (munmap) without affecting others.

Fragmentation

```
608$ ./fragment

Pool: 1024 blocks of 4096 bytes = 4096 KB total
Freed every other block: 512 free blocks (2048 KB free)

Looking for 128 contiguous free blocks (512 KB)...
FAILED - 2048 KB free total, but largest contiguous run < 512 KB
This is EXTERNAL FRAGMENTATION.

--- Internal Fragmentation ---
malloc(1) at 0x5625428b86b0, malloc(1) at 0x5625428b86d0
Distance between them: 32 bytes (wasted: 31 bytes per alloc)

608$
```

Q7.1

Total free memory = 2048 KB, but the largest contiguous block is only 4 KB. The 512 KB request fails because free space is fragmented into small non-contiguous chunks (external fragmentation).

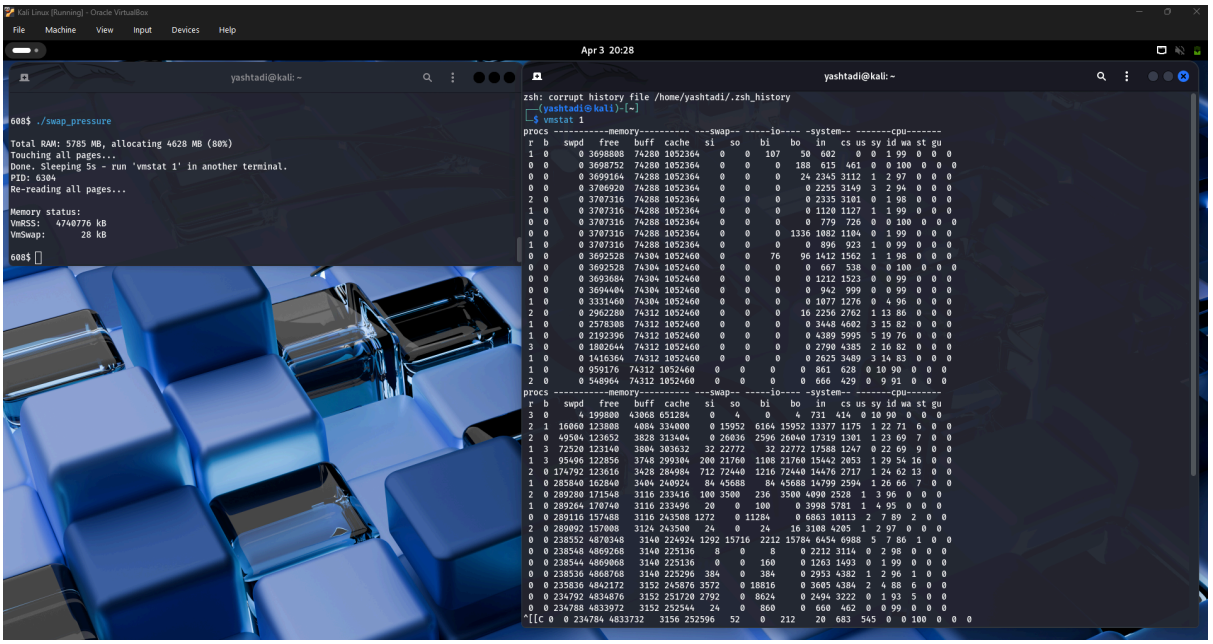
Q7.2

The checkerboard pattern ensures no two free blocks are adjacent. Since coalescing only merges adjacent free blocks, fragmentation persists and large contiguous space never forms.

Q7.3

Paging removes the need for contiguous physical memory by mapping virtual pages to any free physical frames. So a 512 KB allocation can use scattered frames, avoiding external fragmentation at the physical level.

Swapping



Q8.1

Yes, si and so become non-zero when memory pressure increases. They spike when the program touches many pages and the kernel starts swapping pages in/out to free RAM.

Q8.2

If VmSwap is nonzero, it means some pages were moved to disk. The kernel swaps them out because they were least recently used (LRU) and needed free RAM for active pages.

Q8.3

Major page faults are extremely slow (disk access), while minor faults are fast (RAM only). Databases rely on low latency, so swapping causes huge performance drops, making it critical to keep working data in RAM.

Page Replacement

```
608$ grep -E 'Active|Inactive' /proc/meminfo

Active:           392560 kB
Inactive:         585572 kB
Active(anon):     227644 kB
Inactive(anon):   468008 kB
Active(file):     164916 kB
Inactive(file):   117564 kB

608$ nano working_set.c

608$ gcc -O0 -o working_set working_set.c


608$ ./working_set 512 64 5
Total region: 512 MB, working set: 64 MB, iterations: 5

  iter 0: 0.0005 s  (64 MB scanned)
  iter 1: 0.0003 s  (64 MB scanned)
  iter 2: 0.0003 s  (64 MB scanned)
  iter 3: 0.0003 s  (64 MB scanned)
  iter 4: 0.0003 s  (64 MB scanned)

608$ ./working_set 512 512 5
Total region: 512 MB, working set: 512 MB, iterations: 5

  iter 0: 0.0018 s  (512 MB scanned)
  iter 1: 0.0016 s  (512 MB scanned)
  iter 2: 0.0017 s  (512 MB scanned)
  iter 3: 0.0016 s  (512 MB scanned)
  iter 4: 0.0017 s  (512 MB scanned)

608$
```

Q9.1

The 64 MB working set is much faster because it fits in RAM, so most accesses hit cached pages. The 512 MB case causes frequent evictions and page faults, slowing it down due to memory pressure.

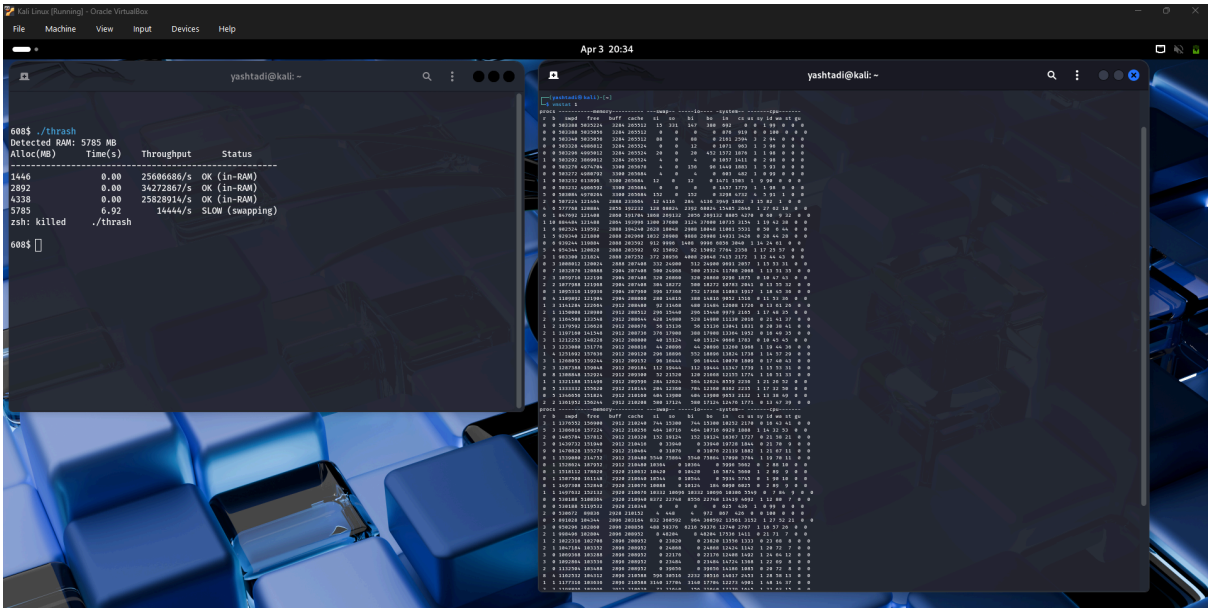
Q9.2

After the run, frequently accessed pages move to the Active list, while less-used ones shift to Inactive. Larger working sets increase inactive pages and churn, showing more page replacement activity.

Q9.3

True LRU requires updating metadata on every memory access, which is too expensive. Linux uses the accessed bit with a second-chance algorithm to approximate LRU efficiently with minimal overhead.

Thrashing



Q10.1

Throughput typically collapses when allocation approaches or exceeds ~100% of RAM. At 75% it remains stable, but around 100–130% it drops sharply due to heavy swapping and thrashing.

Q10.2

si and so spike heavily as pages are constantly swapped in and out. wa (I/O wait) also increases significantly because the CPU is stalled waiting for disk operations.

Q10.3

(a) The OOM killer terminates processes when memory is critically low to prevent system freeze.

(b) vm.swappiness controls how aggressively Linux swaps (0 = avoid swap, 100 = swap aggressively).